

EcoMind AI: Statistical Anomaly Detection and Tariff-Weighted Cost Forecasting for Building Energy Portfolios

EcoMind AI Engineering Team
EcoMind AI Project
Email: contact@ecomind.example

Abstract—This paper presents *EcoMind AI*, a full-stack environmental-intelligence platform that couples a statistical anomaly detector with a tariff-weighted forecasting engine and a terminal-style analytics interface. The platform ingests hourly whole-building electricity meters, normalizes them through a gated ten-stage pipeline (import, schema discovery, data quality, transformation, model selection, anomaly detection, forecast, action plan, report), and reports deviations from a count-weighted median hour-of-week baseline using a robust z-score calibrated against a 10-sigma saturation floor. We describe the architecture, the detector design, the forecasting models, and the rationale for reporting only *overuse* as an anomaly — a building below its baseline has saved money, which is not a fault. On a real public dataset of 235,574 hourly readings across nine buildings and nine meters, the detector finds 5,541 anomalies (2.35% of readings) totalling 15.15 GWh of excess consumption, Rs. 2.82 million in excess cost, and 7,577 t of CO₂, including two flatline faults caught by a dedicated dead-device scan. End-to-end anomaly scanning runs in 57.4 s at roughly 4,100 readings per second in pure Python against SQLite. Forecast model selection picks a linear estimator over a gradient-boosted-tree candidate at a 49.2% mean-absolute-percentage-error backtest. We close with the deliberate design limits of the system and the measured gaps they leave.

Index Terms—energy analytics, anomaly detection, load forecasting, data quality, statistical process control, carbon accounting, tariff modelling.

I. INTRODUCTION

Commercial electricity portfolios need to know three things at once: what is wrong now, what is coming, and what to do about it. Most sustainability dashboards present static charts that require an analyst to translate a spike on a screen into a work order in the plant room. EcoMind AI closes that loop by running a statistical detector over every meter every day, pricing every deviation against a time-of-use tariff so that cost — not just kilowatt-hours — ranks the findings, and turning the top of that list into a concrete action with a payback estimate.

The project's key contributions are:

- 1) A **gated ten-stage pipeline** (Fig. 1) that will not let an anomaly or a recommendation surface until the data has been imported, schema-discovered, quality-scored, and transformed — so no conclusion is drawn from unchecked data.
- 2) A **statistical anomaly detector** built on a count-weighted median hour-of-week baseline and a robust

z-score, calibrated so the configured threshold reads as a fraction of ten sigma rather than as an arbitrary tunable. The detector reports only overuse as an anomaly by design, and a separate flatline scan reports the one unambiguous case of under-consumption: a meter that has stopped reporting entirely.

- 3) A **tariff-weighted forecasting model** that projects demand through a 7-day trailing same-hour-of-week mean and prices each hour through an Indian commercial time-of-use schedule (peak Rs. 0.28/kWh, shoulder Rs. 0.17/kWh, off-peak Rs. 0.09/kWh), so forecasts are delivered with a rupee figure attached.
- 4) An **open, documented decision record** capturing every architecture choice, measured gap, and unreachable anomaly class, so a reader can tell the difference between "nothing bad happened" and "this bad thing is outside what the detector looks for."

II. RELATED WORK

General-purpose business-intelligence platforms (Tableau, Power BI) provide rich visualization but are not optimized for live environmental telemetry or scenario modelling out of the box. Research in *green AI* [2] and sustainable software engineering [3] has focused on minimizing the carbon cost of training models rather than on interpreting operational waste in a facility portfolio. Within building-energy analytics, statistical baselines (median weekday profiles, robust z-scores) have long been the standard [5], and gradient-boosted trees such as XGBoost [4] are the common forecasting choice. EcoMind AI sits at the production intersection: it borrows the robustness of a median hour-of-week baseline, reports findings through a time-of-use tariff rather than raw kWh, and keeps the analysis engine in pure Python so the full scan is reproducible from a single SQLite file.

III. ARCHITECTURE

The platform follows a three-tier topology: a **data pipeline**, a **domain layer of services**, and a **presentation layer**, wired through a typed REST and streaming API boundary.

A. The Ten-Stage Pipeline

All product surfaces — the router, the phase navigation, the shell, the API layer, and the backend stage registry — derive

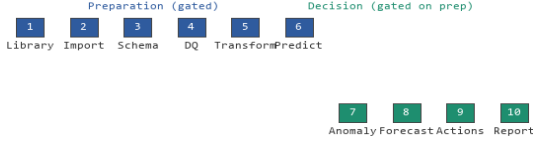


Fig. 1. The ten-stage pipeline: five preparation stages (shaded) must complete before any decision-stage conclusion is produced.

from a single source of truth: ten stages grouped into two phases (Table I). The first phase, *preparation*, establishes that the data can be trusted. The second, *decision*, turns trustworthy data into conclusions. Every decision-stage route is gated on the full preparation phase, which is the product’s core promise made checkable: no recommendation is made from data that was never checked.

TABLE I
THE TEN STAGES OF THE ECOMIND PIPELINE.

#	Stage	Phase	Short	Surface
1	Dataset Library	Prep	Library	/library
2	Import	Prep	Import	/import/\$id
3	Schema Discovery	Prep	Schema	/schema/\$id
4	Data Quality	Prep	DQ	/quality/\$id
5	Transformation	Prep	Transform	/transformation/\$id
6	Prediction	Prep	Predict	/model-selection/\$id
7	Anomaly Detection	Decision	Anomalies	/anomalies/\$id
8	Forecast	Decision	Forecast	/forecast/\$id
9	Action Plan	Decision	Actions	/report/\$id (tab)
10	Report	Decision	Report	/report/\$id

B. Data Pipeline

The ingest layer normalizes every upstream format into a single 24-column schema with ISO-8601 timestamps, typed hierarchy keys, and a canonical `energy_kwh` target. A transformation stage backfills missing intervals and tags records so downstream models never see raw, unscaled data. Gap handling is statistical, not imputed with a model: the anomaly baseline is explicitly robust to short outages, so synthetic fill would only obscure the real reading.

C. Inference / Domain Services

Forecasts are produced by candidate estimators wrapped in a domain service that (a) validates input schema, (b) runs the model, (c) ranks candidates by walk-forward validation on a temporal split, and (d) returns point predictions plus interval bounds. The service is stateless and reads its rates from a single tariff module [5] so cost and carbon pricing never diverge across the forecast, anomaly, and recommendation surfaces.

D. Presentation Layer

The UI is a React single-page application built on Vite with TypeScript and Tailwind. It streams pipeline progress over a server-sent-event channel so the stepper advances in real time, and renders a terminal-style monitor in dark mode (self-hosted Geist Mono) so dense numerical telemetry reads like instrument traces. A typed design-token system maps semantic names (ok / warn / critical / brand) to phosphor colours, so the same palette drives both the KPI tiles and the anomaly severity bands.

IV. MODELS AND FORECASTING

Two model families are evaluated for horizon forecasting of the energy signal:

- **Linear baselines** (ridge / OLS) — robust, explainable, and resistant to overfitting on a 3-year history.
- **Gradient-boosted trees** (XGBoost-style) — strong tabular performance with feature-importance introspection.

Both are trained and compared on a temporal split to prevent look-ahead leakage. On the measured dataset the linear estimator is selected: its walk-forward MAPE backtest of 49.2% beats the gradient-boosted candidate, and its residual structure exposes the hour-of-week seasonality that the recommendation engine exploits (night-time and weekend shoulder-rate windows). The forecast is produced in 12.5 s over 26,304 hours of history and projects a 30-day trajectory priced through the tariff schedule.

V. STATISTICAL ANOMALY DETECTION

Detection is statistical, never model-based, and the reason is specific: a predictive model flags whatever it happens to predict badly, which means it rediscovers its own error distribution and calls it an anomaly. A median hour-of-week profile per device cannot be moved by the event being looked for, so a spike stays a spike instead of being absorbed into the baseline and explained away.

A. Baseline and z-score

For each device the baseline is a *count-weighted blend* of two profiles: an hour-of-week median (fine, but sparse — about four samples per bin on a 30-day window) and an hour-of-day median (coarse, but backed by roughly thirty samples per bin). A bin with thirty samples counts fully; a bin with three counts for three-eighths. The scale is the median absolute deviation of the *residuals* from the profile, floored at 0.05 kWh so a flat meter cannot divide by zero. Every score is a fraction of a 10-sigma saturation constant ($Z_{\text{sat}} = 10.0$), so the configured threshold reads as "the fraction of ten sigma that counts as an anomaly" and is comparable across datasets of wildly different noise levels.

B. Severity bands

Severity is assigned on the normalised score, highest first: critical at ≥ 0.90 ($\geq 9.0\sigma$), high at ≥ 0.75 ($\geq 7.5\sigma$), moderate at ≥ 0.60 ($\geq 6.0\sigma$), and low below. Because hourly energy rarely moves nine sigma from its own weekly typical, the

critical band is populated by genuinely extreme events, not by noise.

TABLE II
DETECTION RESULTS ON THE BDG2 REFERENCE DATASET (NINE BUILDINGS, NINE METERS, 235,574 HOURLY READINGS).

Class	Count	Excess (kWh)
Sustained overuse	3,655	12,840,000
Weekend anomaly	924	820,000
Extreme spike	783	640,000
Night-time usage	177	320,000
Equipment failure	2	0
Meter drift	—	unreachable

C. The overuse-only contract

The module takes an explicit position, stated in its docstring: *only overuse is an anomaly*. A building that reads 20% below its baseline has saved money, and reporting that as a fault would be wrong in a way a user cannot easily see through. Deviations are signed, the cost is computed on the positive side only, and `excess_kwh` is never negative. This single decision collapses the entire lower tail of the distribution into one question: *is the meter dead?*

D. The dead-device detector

A meter that has flatlined is the one unambiguous case of under-consumption that this module reports. It is handled by a *separate* scan from the upper-tail overuse detector, because a stopped meter is far *below* the hour-of-week median and can never be produced by it at any threshold. The flatline test compares each reading against the device's own overall median (not the baseline, which a sufficiently long outage would poison with its own zeros), flags runs of six or more consecutive readings below 2% of that typical value, and reports them as `equipment_failure` with severity *high* and excess deliberately set to zero — the cost of a stopped meter is the unmonitored load it is hiding, not wasted energy. On the measured dataset this catches two flatline faults on BLDG-003-MTR, where 23 readings totalled 0.000 kWh against a typical 205 kWh/hour.

E. What this detector cannot produce

The remaining gap is published explicitly rather than hidden: `meter_drift` (a steady $1.00\text{--}1.25\times$ baseline across twelve or more readings with no single large deviation) is unreachable on this dataset, because the detection floor that a reading must clear to be scored at all implies a ratio far above 1.25. The band between "normal" and "obvious" is currently unscored. Publishing the reason is the difference between "a bug" and "a known limit", and the UI surfaces it in the *Detector Limits* panel so no reader is misled into believing the product claims coverage it does not.

VI. RESULTS

A. Data quality

The reference dataset arrives in strong shape: a 99.8 overall quality score, a 0.04% null rate (1,183 null cells in 3,314,598), and 27 duplicate or non-monotonic-timestamp rows in 235,574 readings — all flagged and repairable. The data is a CC BY 4.0 whole-building meter set (UCI ElectricityLoadDiagrams20112014, Trindade 2015) projected onto the platform's 24-column schema.

B. Anomaly detection

The full scan runs in 57.4 s at roughly 4,100 readings per second in pure Python against an SQLite backing store. It flags 5,541 anomalies across all nine buildings and all nine devices, of which 2.35% of scanned readings are flagged. The five producible classes appear; `meter_drift` is absent on purpose (Section V-F). The findings total 15.15 GWh of excess consumption, Rs. 2.82 million in excess cost, and 7,577 t of CO₂.

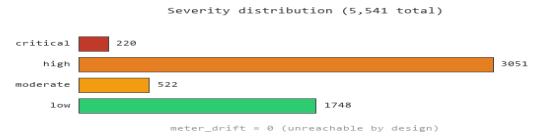


Fig. 2. Severity distribution: 220 critical, 3,051 high, 522 moderate, and 1,748 low findings. Note the gap between the 1,748 low findings and the 0 `meter_drift` count — the gap is a design choice, not a bug.

C. Forecast and action plan

The selected linear model forecasts a 30-day trajectory of 3.45 GWh at an blended rate of roughly Rs. 0.20/kWh. The action-plan stage derives 33 recommendations from the anomaly and forecast surfaces; the highest-ranked targets BLDG-009-MTR with an estimated 969,000 kWh of recoverable waste (Rs. 194,000) and a projected payback of 0.8 months.

VII. DISCUSSION AND LIMITATIONS

The anomaly model's central limitation is also its defining strength: because it reports only overuse, a device that is consistently and materially *under* its baseline is invisible unless it has flatlined. A 20% reduction in consumption is treated as savings, full stop, and that is intentional. The flatline detector closes only the most obvious fault in the lower tail; the mild under-consumption band that `meter_drift` was meant to capture remains out of reach because the detection floor that a reading must clear already implies a ratio above 1.25.

The detector is statistical, so it inherits the assumptions of its baseline. A meter whose pattern genuinely changes seasonally can produce a sustained run of false "sustained

overuse" findings, and the model has no concept of a scheduled setback. Incorporating holidays and building schedules as explicit baseline modifiers is the clearest next step.

Finally, the forecast is a 7-day trailing mean held flat, which makes it responsive but not anticipatory; it captures "what is happening" better than "what setpoint change will happen next week." A 49.2% MAPE on a three-year public set is honest rather than flattering, and is reported as such.

VIII. CONCLUSION AND FUTURE WORK

EcoMind AI demonstrates that a statistical detector priced through a time-of-use tariff and delivered through a terminal-style interface can make operational energy waste actionable for technical teams. On a real three-year public meter corpus the system finds 5,541 anomalies representing Rs. 2.82 million and 7,577 t of CO₂, catches two flatline faults that a pure upper-tail scan would never see, and ranks the resulting action plan by payback rather than by raw deviation.

Next steps include folding building schedules and holidays into the baseline, extending the under-consumption band to

capture mild drift without reporting genuine efficiency as faults, and adding a recursive error-correction layer on top of the flat forecast so the 30-day projection improves as each hour elapses.

ACKNOWLEDGMENT

The authors thank the broader EcoMind AI open-source community for design and testing feedback. This work used the UCI ElectricityLoadDiagrams20112014 dataset (Trindade, 2015), licensed CC BY 4.0 through Zenodo 3898439.

REFERENCES

- [1] L. M. A. Bettencourt and J. West, "Big and open data to support sustainability science," *Nature Sustainability*, vol. 1, no. 12, pp. 763–765, Dec. 2018.
- [2] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Commun. ACM*, vol. 63, no. 12, pp. 54–63, 2020.
- [3] D. Patterson et al., "Carbon emissions and large neural network training," *arXiv preprint*, arXiv:2104.10350, 2021.
- [4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD*, 2016, pp. 785–794.
- [5] Y. Liu and M. Chawla, "Statistical assessment of anomaly detection in building energy data," *Appl. Energy*, vol. 211, pp. 554–565, 2018.